US Patent & Trademark Office
Patent Public Search | Text View

United States Patent	12386783
Kind Code	B2
Date of Patent	August 12, 2025
Inventor(s)	George; Tijin et al.

Snapshot storage and management within an object store

Abstract

Techniques are provided for an object file system for an object store. Data, maintained by a computing device, is stored into slots of an object. The data within the slots of the object is represented as a data structure comprising a plurality of nodes comprising cloud block numbers used to identify the object and particular slots of the object. A mapping metafile is maintained to map block numbers used to store the data by the computing device to cloud block numbers of nodes representing portion of the data stored within slots of the object. The object is stored into the object store, and the mapping metafile and the data structure are used to provide access through the object file system to portions of data within the object.

Inventors: **George; Tijin (Sunnyvale, CA), Nehra; Jagavar (Bangalore, IN), Chuggani; Roopesh (Bengaluru, IN), Pawar; Dnyaneshwar Nagorao (Bangalore, IN), Pandit; Atul Ramesh (Los Gatos, CA), Ponnapur; Anil Kumar (Sunnyvale, CA), Mathew; Jose (Santa Clara, CA), Venketaraman; Sriram (Bangalore, IN)**

Applicant: **NetApp Inc.** (San Jose, CA)

Family ID: **72336424**

Assignee: **NetApp, Inc. (San Jose, CA)**

Appl. No.: **18/406338**

Filed: **January 08, 2024**

Prior Publication Data

Document Identifier	Publication Date
US 20240143549 A1	May. 02, 2024

Related U.S. Application Data

continuation parent-doc US 17498263 20211011 US 11868312 child-doc US 18406338
continuation parent-doc US 16401294 20190502 US 11144503 20211012 child-doc US 17498263

continuation parent-doc US 16296417 20190308 US 11144502 20211012 child-doc US 16401294

Publication Classification

Int. Cl.: **G06F16/14** (20190101); **G06F11/14** (20060101); **G06F16/11** (20190101); **G06F16/182** (20190101)

U.S. Cl.:

CPC **G06F16/148** (20190101); **G06F11/1451** (20130101); **G06F16/128** (20190101); **G06F16/182** (20190101); G06F2201/80 (20130101); G06F2201/84 (20130101)

Field of Classification Search

CPC: G06F (16/148); G06F (16/182); G06F (16/128); G06F (11/1451); G06F (2201/80); G06F (2201/84)

USPC: 707/624

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
8533410	12/2012	Corbett et al.	N/A	N/A
9367551	12/2015	Beaverson et al.	N/A	N/A
9798496	12/2016	Varghese et al.	N/A	N/A
11144502	12/2020	George et al.	N/A	N/A
11144503	12/2020	George et al.	N/A	N/A
11868312	12/2023	George et al.	N/A	N/A
2015/0269032	12/2014	Muthyala	707/649	G06F 16/178
2016/0210306	12/2015	Kumarasamy	N/A	G06F 16/252
2016/0283501	12/2015	König	N/A	G06F 16/11
2017/0123935	12/2016	Pandit et al.	N/A	N/A
2018/0124174	12/2017	Swallow	N/A	H04L 67/60
2018/0196831	12/2017	Maybee	N/A	G06F 11/07
2020/0183602	12/2019	Kabra	N/A	G06F 3/067
2022/0027313	12/2021	George et al.	N/A	N/A

OTHER PUBLICATIONS

Perry Y., "Storage Snapshots Deep-Dive: Cloud Volumes Snapshots," Jun. 28, 2018, pp. 1-4. URL: https://cloud.netapp.com/blog/snapshots-technology-cloud-volumes. cited by applicant

Stender J., "Snapshots in Large-Scale Distributed File Systems," Sep. 14, 2012, pp. 1-139. URL: https://d-nb.info/1030313644/34. cited by applicant

Primary Examiner: Bullock; Joshua

Attorney, Agent or Firm: Cooper Legal Group, LLC

Background/Summary

(1) This application claims priority to and is a continuation of U.S. patent application Ser. No. 17/498,263, titled “SNAPSHOT STORAGE AND MANAGEMENT WITHIN AN OBJECT STORE” and filed on Oct. 11, 2021, which claims priority to and is a continuation of U.S. Pat. No. 11,144,503, titled “SNAPSHOT STORAGE AND MANAGEMENT WITHIN AN OBJECT STORE” and filed on May 2, 2019, which claims priority to and is a continuation of U.S. patent application Ser. No. 16/296,417, titled “OBJECT STORE FILE SYSTEM FORMAT FOR REPRESENTING, STORING, AND RETRIEVING DATA IN AN OBJECT STORE ACCORDING TO A STRUCTURED FORMAT” and filed on Mar. 8, 2019, which are incorporated herein by reference.

BACKGROUND

(1) Many users utilize cloud computing environments to store data, host applications, etc. A client device may connect to a cloud computing environment in order to transmit data from the client device to the cloud computing environment for storage. The client device may also retrieve data from the cloud computing environment. In this way, the cloud computing environment can provide scalable low cost storage.

(2) Some users and businesses may use or deploy their own primary storage systems such as clustered networks of nodes (storage controllers) for storing data, hosting applications, etc. A primary storage system may provide robust data storage and management features, such as data replication, data deduplication, encryption, backup and restore functionality, snapshot creation and management functionality, incremental snapshot creation, etc. However, storage provided by such primary storage systems can be relatively more costly and less scalable compared to cloud computing storage. Thus, cost savings and scalability can be achieved by using a hybrid of primary storage systems and remote cloud computing storage. Unfortunately, the robust functionality provided by primary storage systems is not compatible with cloud computing storage, and thus these features are lost.

Description

DESCRIPTION OF THE DRAWINGS

(1) FIG. **1** is a component block diagram illustrating an example clustered network in which an embodiment of the invention may be implemented.

(2) FIG. **2** is a component block diagram illustrating an example data storage system in which an embodiment of the invention may be implemented.

(3) FIG. **3** is a flow chart illustrating an example method for managing objects within an object store using an object file system.

(4) FIG. **4**A is a component block diagram illustrating an example system for managing objects within an object store using an object file system.

(5) FIG. **4**B is an example of a snapshot file system within an object store.

(6) FIG. **4**C is an example of an object stored within an object store.

(7) FIG. **5** is an example of a computer readable medium in which an embodiment of the invention may be implemented.

(8) FIG. **6** is a component block diagram illustrating an example computing environment in which an embodiment of the invention may be implemented.

DETAILED DESCRIPTION

(9) Some examples of the claimed subject matter are now described with reference to the drawings, where like reference numerals are generally used to refer to like elements throughout. In the

following description, for purposes of explanation, numerous specific details are set forth in order to provide an understanding of the claimed subject matter. It may be evident, however, that the claimed subject matter may be practiced without these specific details. Nothing in this detailed description is admitted as prior art.

(10) As provided herein, an object file system is provided that is used to store, retrieve, and manage objects within an object store, such as a cloud computing environment. The object file system is capable of representing data in the object store in a structured format. It may be appreciated that any type of data (e.g., a file, a directory, an image, a storage virtual machine, a logical unit number (LUN), application data, backup data, metadata, database data, a virtual machine disk, etc.) residing in any type of computing device (e.g., a computer, a laptop, a wearable device, a tablet, a storage controller, a node, an on-premise server, a virtual machine, another object store or cloud computing environment, a hybrid storage environment, data already stored within the object store, etc.) using any type of file system can be stored into objects for storage within the object store. This allows the data to be represented as a file system so that the data of the objects can be accessed and mounted on-demand by remote computing devices. This also provides a high degree of flexibility in being able to access data from the object store, a cloud service, and/or a network file system for analytics or data access on an on-demand basis. The object file system is able to represent snapshots in the object store, and provides the ability to access snapshot data universally for whomever has access to an object format of the object file system. Snapshots in the object store are self-representing, and the object file system provides access to a complete snapshot copy without having to access other snapshots.

(11) The object file system provides the ability to store any number of snapshots in the object store so that cold data (e.g., infrequently accessed data) can be stored for long periods of time in a cost effective manner, such as in the cloud. The object file system stores data within relatively larger objects to reduce cost. Representation of data in the object store is complete, such that all data and required container properties can be independently recovered from the object store. The object file system format ensures that access is consistent and is not affected by eventual consistent nature of underlying cloud infrastructure.

(12) The object file system provides version neutrality. Changes to on-prem metadata versions provide little impact on the representation of data in the object store. This allow data to be stored from multiple versions of on-prem over time, and the ability to access data in the object store without much version management. The object file system provides an object format that is conducive to garbage collection for freeing objects (e.g., free slots and/or objects storing data of a delete snapshot), such as where a lower granularity of data can be garbage collected such as at a per snapshot deletion level.

(13) In an embodiment, snapshots of data, such as of a primary volume, maintained by a computing device (e.g., a node, storage controller, or other on-prem device that is remote to the object store) can be created by the computing device. The snapshots can be stored in the object store independent of the primary volume and can be retained for any duration of time. Data can be restored from the snapshots without dependency on the primary volume. The snapshot copies in the object store can be used for load distribution, development testing, virus scans, analytics, etc. Because the snapshot copies (e.g., snapshot data stored within objects) are independent of the primary volume at the computing device, such operations can be performed without impacting performance of the computing device.

(14) A snapshot is frozen in time representation of a filesystem. All the necessary information may be organized as files. All the blocks of the file system may be stitched together using cloud block numbers (e.g., a cloud block number comprises a sequence number of an object and a slot number of a slot within that object) and the file will be represented by a data structure (e.g., represented in a tree format of a tree structure) when stored into the object store within one or more objects. Using cloud block numbers, a next node within the tree structure can be identified for traversing the tree

structure to locate a node representing data to be accessed. The block of the data may be packed into bigger objects to be cloud storage friendly, where blocks are stored into slots of a bigger object that is then stored within the object store. All the indirections (pointers) to reach leaf nodes of a file (e.g., user data such as file data is represented by leaf nodes within the tree structure) may be normalized and may be version independent. Every snapshot may be a completely independent copy and any data for a snapshot can be located by walking the object file system. While doing incremental snapshot copy, changed blocks between two snapshots may be copied to the object store, and unchanged blocks will be shared with previous snapshots as opposed to being redundantly stored in the object store. In this way, deduplication is provided for and between snapshot data stored within objects of the object store. As will be described later, an embodiment of a snapshot file system in the object store is illustrated by FIG. **4**B.

(15) Cloud block numbers are used to uniquely represent data (e.g., a block's worth of information from the computing device) in the object store at any point in time. A cloud block number is used to derive an object name (e.g., a sequence number) and an index (a particular slot) within the object. An object format, used by the object file system to format objects, allows for sharing of cloud blocks. This provides for storage space efficiency across snapshots so that deduplication and compression used by the computing device will be preserved. Additional compression is applied before writing objects to the object store and information to decompress the data is kept in the object header.

(16) Similar to data (e.g., a file, directory, or other data stored by the computing device), metadata can be stored into objects. Metadata is normalized so that the restoration of data using the metadata from an object to a remote computing device will be version independent. That is, snapshot data at the computing device can be stored into objects in a version neutral manner. Snapshots can be mounted and traversed independent of one another, and thus data within an object is represented as a file system, such as according to the tree structure. The format of non-leaf nodes of the tree structure (e.g., indirects such as pointers to other non-leaf nodes or to leaf nodes of user data) can change over time. In this way, physical data is converted into a version independent format as part of normalization. Denormalization may be performed while retrieving data from the objects, such as to restore a snapshot. In an example of normalization, a slot header in an object has a flag that can be set to indicate that a slot comprises normalized content. Each slot of the object is independently represented. Slot data may comprise version data. The slot data may specify a number of entries within the object and an entry size so that starting offsets of a next entry can be calculated from the entry size of a current entry.

(17) In an embodiment, denormalization of a first version of data/metadata (e.g., a prior version) can be retrieved from data backed up in an object according to a second version (e.g., a future version). In an example, if the future version added a new field, then during denormalization, the new field is skipped over. Denormalization of a future version can be retrieved from data backed up in an object according to a prior version. A version indicator in the slot data can be used to determine how of an entry is to be read and interpreted, and any missing fields will be set to default values.

(18) In an embodiment of the object format of objects stored within the object store, relatively larger objects will be stored in the object store. As will be described later, an embodiment of an object is illustrated by FIG. **4**C. An object comprises an object header followed by data blocks (slots). The object header has a static array of slot context comprising information used to access data for slots. Each slot can represent any length of logical data (e.g., a slot is a base unit of data of the object file system of the object store). Since data blocks for metadata are normalized, a slot can represent any length of logical data. Data within the slots can be compressed into compression groups, and a slot will comprise enough information for how to decompress and return data of the slot.

(19) In an embodiment, storage efficiency provided by the computing device is preserved within

the object store. A volume copied from the computing device into objects of the object store is maintained in the object store as an independent logical representation of the volume. Any granularity of data can be represented, such as a directory, a qtree, a file, or other data container. A mapping metafile (a VMAP) is used to map virtual block IDs/names (e.g., a virtual volume block number, a hash, a compression group name, or any other set of names of a collection of data used by the computing device) to cloud block numbers in the object store. This mapping metafile can be used to track duplicate data per data container for storage efficiency.

(20) The mapping metafile enables duplicate data detection of duplicate data, such as a duplicate block or a compression group (e.g., a compressed group of blocks/slots within an object). The mapping metafile is used to preserve sharing of data within and across multiple snapshots stored within objects in the object store. The mapping metafile is used for sharing of groups of data represented by a unique name. The mapping metafile is used to populate indirect blocks with corresponding cloud block numbers for children nodes (e.g., compressed or non-compressed). The mapping metafile is used to help a garbage collector make decisions on what cloud block numbers can be freed from the object store when a corresponding snapshot is deleted by the computing device. The mapping metafile is updated during a snapshot copy operation to store snapshot data from the computing device into objects within the object store. An overflow mapping metafile can also be used, such as to represent entries with base key collision. The overflow mapping metafile will support variable length key and payload in order to optimize a key size according to a type of entry in the overflow mapping metafile.

(21) The mapping metafile may be indexed by virtual volume block numbers or starting virtual volume block numbers of a compression group. An entry within the mapping metafile may comprise a virtual volume block number as a key, a cloud block number, an indication of whether the cloud block number is the start of a compression group, a compression indicator, an indicator as to whether additional information is stored in the overflow mapping metafile, a logical length of the compression group, a physical length of the compression group, etc. Entries are removed/invalidated from the mapping metafile if corresponding virtual volume block numbers are freed by the computing device, such as when a snapshot is deleted by the computing device.

(22) The data structure, such as the tree structure, is used to represent data within an object. Each node of the tree structure is represented by a cloud block number. The key to the tree structure may uniquely identify uncompressed virtual volume block numbers, a contiguous or non-contiguous compression group represented by virtual volume block numbers associated with such, and/or an entry for non-starting virtual volume block numbers of the compression group to a starting virtual volume block number of the compression group. A key will comprise a virtual volume block number, a physical length of a compression group, an indicator as to whether the entry represents a start of the compression group, and/or a variable length array of virtual volume block numbers of either non-starting virtual volume block numbers or the starting virtual volume block number (if uncompressed then this is field is not used). The payload will comprise cloud block numbers and/or flags corresponding to entries within the mapping metafile.

(23) Before transferring objects to the object store for an incremental snapshot, the mapping metafile is processed to clear any stale entries. This is to ensure that a stale virtual volume block number or compression group name is not reused for sharing (deduplication). In particular, between two snapshots, all virtual volume block numbers transitioning from a 1 to 0 (to indicate that the virtual volume block numbers are no longer used) in a snapshot to be copied to the object store in one or more objects are identified. Entries within the mapping metafile for these virtual volume block numbers transitioning from a 1 to 0 are removed from the mapping metafile. In this way, all entries using these virtual volume block numbers are invalidated.

(24) As part of copying a snapshot to the object store, changed data and indirections for accessing the changed data are transferred (or all data for initialization). In particular, changed user data of the computing device is traversed through buftrees using a snapdiff operation to determine a data

difference between two snapshots. Logical (uncompressed) data is read and populated into objects and associated with cloud block numbers. To preserve storage efficiency, a mapping from a unique name representing the logical data (e.g., virtual volume block number or a compression group name for compressed data) to a cloud block number (e.g., of a slot within which the logical data is stored) is recorded in the mapping metafile. Lookups to the mapping metafile will be performed to ensure only a single copy of changed blocks are copied to the object store. Metadata is normalized for version independency and stored into objects. Indirects (non-leaf nodes) are stored in the object to refer to unchanged old cloud blocks and changed new cloud blocks are stored in the object, which provides a complete view of user data and metadata for each snapshot. Inodes are written to the object store while pushing changed inofile blocks to the object store. Each inode entry within an inofile is normalized to represent a version independent inode format. Each inode will have a list of next level of indirect blocks (e.g., non-leaf nodes of the tree structure storing indirects/pointers to other nodes). Snapinfo objects comprise snapshot specific information. A snapinfo object of a snapshot has a pointer to a root of a snapshot logical file system. A root object for each primary volume (e.g., a primary volume for which a snapshot is captured) is copied to the object store. Each snapshot is associated with an object ID (sequence number) map that tracks which objects are in use in a snapshot (e.g., which objects comprise data of the snapshot) and is subsequently used for garbage collection in the future when a particular snapshot is deleted.

(25) In an embodiment of data access and restoration, the tree format represents an object file system (a cloud file system) that can be mounted and/or traversed from any remote device utilizing APIs using a thin layer orchestrating between client requests and object file system traversal. A remote device provides an entry point to the object tree using a universal identifier (UUID) that is a common identifier for all object names for a volume (or container). A rel root object is derived from the UUID, which has pointers (names) to next level snapinfo objects. If a user is browsing a snapshot, a snapshot snapinfo is looked up within snapinfo objects. If no snapshot is provided, then latest snapshot info is used. The snapshot info has cloud block numbers for an inode file. The inode file is read from the object store using the cloud block number and an inode within the inode file is read by traversing the inode file's tree structure. Each level including the inode has a cloud block number for a next level until a leaf node (a level 0 block of data) is read. Thus, the inode for the file of interest is obtained, and the file's tree structure is traversed by looking up cloud block number for a next level of the tree structure (e.g., a cloud block number from a level 1 is used to access the level 0 block) until the required data is read. Object headers and higher level indirects are cached to reduce the amount of access to the object store. Additionally, more data may be read from the object store than needed to benefit from locality for caching. Data access can be used to restore a complete copy of a snapshot, part of a snapshot (e.g., a single file or directory), or metadata.

(26) In an embodiment of read/write cloning, a volume or file, backed from a snapshot in the object store, is created. Read access will use a data access path through a tree structure. At a high level, write access will read the required data from the object store (e.g., user data and all levels of the file/volume tree that are part of user data modification by a write operation). The blocks are modified and the modified content is rewritten to the object store.

(27) In an embodiment, defragmentation is provided for objects comprising snapshot copies in the object store and to prevent fragmented objects from being sent to the object store during backup. Defragmentation of objects involves rewriting an object with only used data, which may exclude unused/freed data no longer used by the computing device (e.g., data of a deleted snapshot no longer referenced by other snapshots). An object can only be overwritten if used data is not changed. Object sequence numbers are not reused. Only unused data can be freed, but used data cannot be overwritten. Reads will ensure that slot header and data are read from same object (timestamp checking). Reading data from the object store involves reading the header info and then reading the actual data. If these two reads go to different objects (as determined by timestamp comparison), then the read operation is failed and retried.

(28) Defragmentation occurs when snapshots are deleted and objects could not be freed because another snapshot still contains some reference to the objects that would be freed (not all slots within these objects are freed but some still comprise used data from other snapshots). A slot within an object can only be freed when all snapshots referring to that slot are deleted (e.g., an oldest snapshot having the object in use such that younger snapshots do not reuse the freed slots). Also, ownership count can be persistently stored. When a snapshot is deleted, all objects uniquely owned by that snapshot are freed, but objects present in other snapshots (e.g., a next/subsequent snapshot) are not freed. A count of such objects is stored with a next snapshot so that the next snapshot becomes the owner of those objects. Defragmentation is only performed when a number of used slots in an object (an object refcount) is less than a threshold. If the number is below a second threshold, then further defragmentation is not performed. In order to identify used slots and free slots, the file system in the snapshot is traversed and a bitmap is constructed where a bit will be used to denote if a cloud block is in use (a cloud block in-use bitmap). This map is used to calculate the object refcount.

(29) To perform defragmentation, the cloud block in-use map is prepared by walking the cloud snapshot file system. This bitmap is walked to generate an object refcount for the object. The object refcount is checked to see if it is within a range to be defragmented. The object is checked to see if the object is owned by the snapshot by comparing an object ID map of a current and a previous snapshot. If the object is owned and is to be defragmented, then the cloud block in-use map is used to find free slots and to rewrite the object to comprise data from used slots and to exclude freed slots. The object header will be updated accordingly with new offsets.

(30) Fragmentation may be mitigated. During backup, an object ID map is created to contain a bit for each object in use by the snapshot (e.g., objects storing snapshot data of the snapshot). The mapping metafile (VMAP) is walked to create the object ID map. An object reference map can be created to store a count of a number of cloud blocks in use in that object. If the count is below a threshold, then data of the used blocks can be rewritten in a new object.

(31) For each primary volume copied to the object store, there is a root object having a name starting with a prefix followed by a destination end point name and UUID. The root object is written during a conclude phase. Another copy for the root object is maintained with a unique name as a defense to eventual consistency, and will have a generation number appended to the name. A relationship state metafile will be updated before the root object info is updated. The root object has a header, root info, and bookkeeping information. A snapshot info is an object containing snapshot specific information, and is written during a conclude phase of a backup operation. Each object will have its own unique sequence number, which is generated automatically.

(32) To provide for managing objects within an object store using an object file system, FIG. **1** illustrates an embodiment of a clustered network environment **100** or a network storage environment. It may be appreciated, however, that the techniques, etc. described herein may be implemented within the clustered network environment **100**, a non-cluster network environment, and/or a variety of other computing environments, such as a desktop computing environment. That is, the instant disclosure, including the scope of the appended claims, is not meant to be limited to the examples provided herein. It will be appreciated that where the same or similar components, elements, features, items, modules, etc. are illustrated in later figures but were previously discussed with regard to prior figures, that a similar (e.g., redundant) discussion of the same may be omitted when describing the subsequent figures (e.g., for purposes of simplicity and ease of understanding).

(33) FIG. **1** is a block diagram illustrating the clustered network environment **100** that may implement at least some embodiments of the techniques and/or systems described herein. The clustered network environment **100** comprises data storage systems **102** and **104** that are coupled over a cluster fabric **106**, such as a computing network embodied as a private Infiniband, Fibre Channel (FC), or Ethernet network facilitating communication between the data storage systems **102** and **104** (and one or more modules, component, etc. therein, such as, nodes **116** and **118**, for

example). It will be appreciated that while two data storage systems **102** and **104** and two nodes **116** and **118** are illustrated in FIG. **1**, that any suitable number of such components is contemplated. In an example, nodes **116**, **118** comprise storage controllers (e.g., node **116** may comprise a primary or local storage controller and node **118** may comprise a secondary or remote storage controller) that provide client devices, such as host devices **108**, **110**, with access to data stored within data storage devices **128**, **130**. Similarly, unless specifically provided otherwise herein, the same is true for other modules, elements, features, items, etc. referenced herein and/or illustrated in the accompanying drawings. That is, a particular number of components, modules, elements, features, items, etc. disclosed herein is not meant to be interpreted in a limiting manner.

(34) It will be further appreciated that clustered networks are not limited to any particular geographic areas and can be clustered locally and/or remotely. Thus, In an embodiment a clustered network can be distributed over a plurality of storage systems and/or nodes located in a plurality of geographic locations; while In an embodiment a clustered network can include data storage systems (e.g., **102**, **104**) residing in a same geographic location (e.g., in a single onsite rack of data storage devices).

(35) In the illustrated example, one or more host devices **108**, **110** which may comprise, for example, client devices, personal computers (PCs), computing devices used for storage (e.g., storage servers), and other computers or peripheral devices (e.g., printers), are coupled to the respective data storage systems **102**, **104** by storage network connections **112**, **114**. Network connection may comprise a local area network (LAN) or wide area network (WAN), for example, that utilizes Network Attached Storage (NAS) protocols, such as a Common Internet File System (CIFS) protocol or a Network File System (NFS) protocol to exchange data packets, a Storage Area Network (SAN) protocol, such as Small Computer System Interface (SCSI) or Fiber Channel Protocol (FCP), an object protocol, such as S3, etc. Illustratively, the host devices **108**, **110** may be general-purpose computers running applications, and may interact with the data storage systems **102**, **104** using a client/server model for exchange of information. That is, the host device may request data from the data storage system (e.g., data on a storage device managed by a network storage control configured to process I/O commands issued by the host device for the storage device), and the data storage system may return results of the request to the host device via one or more storage network connections **112**, **114**.

(36) The nodes **116**, **118** on clustered data storage systems **102**, **104** can comprise network or host nodes that are interconnected as a cluster to provide data storage and management services, such as to an enterprise having remote locations, cloud storage (e.g., a storage endpoint may be stored within a data cloud), etc., for example. Such a node in the clustered network environment **100** can be a device attached to the network as a connection point, redistribution point or communication endpoint, for example. A node may be capable of sending, receiving, and/or forwarding information over a network communications channel, and could comprise any device that meets any or all of these criteria. One example of a node may be a data storage and management server attached to a network, where the server can comprise a general purpose computer or a computing device particularly configured to operate as a server in a data storage and management system.

(37) In an example, a first cluster of nodes such as the nodes **116**, **118** (e.g., a first set of storage controllers configured to provide access to a first storage aggregate comprising a first logical grouping of one or more storage devices) may be located on a first storage site. A second cluster of nodes, not illustrated, may be located at a second storage site (e.g., a second set of storage controllers configured to provide access to a second storage aggregate comprising a second logical grouping of one or more storage devices). The first cluster of nodes and the second cluster of nodes may be configured according to a disaster recovery configuration where a surviving cluster of nodes provides switchover access to storage devices of a disaster cluster of nodes in the event a disaster occurs at a disaster storage site comprising the disaster cluster of nodes (e.g., the first cluster of nodes provides client devices with switchover data access to storage devices of the

second storage aggregate in the event a disaster occurs at the second storage site).
(38) As illustrated in the clustered network environment **100**, nodes **116**, **118** can comprise various functional components that coordinate to provide distributed storage architecture for the cluster. For example, the nodes can comprise network modules **120**, **122** and disk modules **124**, **126**. Network modules **120**, **122** can be configured to allow the nodes **116**, **118** (e.g., network storage controllers) to connect with host devices **108**, **110** over the storage network connections **112**, **114**, for example, allowing the host devices **108**, **110** to access data stored in the distributed storage system. Further, the network modules **120**, **122** can provide connections with one or more other components through the cluster fabric **106**. For example, in FIG. **1**, the network module **120** of node **116** can access a second data storage device by sending a request through the disk module **126** of node **118**.
(39) Disk modules **124**, **126** can be configured to connect one or more data storage devices **128**, **130**, such as disks or arrays of disks, flash memory, or some other form of data storage, to the nodes **116**, **118**. The nodes **116**, **118** can be interconnected by the cluster fabric **106**, for example, allowing respective nodes in the cluster to access data on data storage devices **128**, **130** connected to different nodes in the cluster. Often, disk modules **124**, **126** communicate with the data storage devices **128**, **130** according to the SAN protocol, such as SCSI or FCP, for example. Thus, as seen from an operating system on nodes **116**, **118**, the data storage devices **128**, **130** can appear as locally attached to the operating system. In this manner, different nodes **116**, **118**, etc. may access data blocks through the operating system, rather than expressly requesting abstract files.
(40) It should be appreciated that, while the clustered network environment **100** illustrates an equal number of network and disk modules, other embodiments may comprise a differing number of these modules. For example, there may be a plurality of network and disk modules interconnected in a cluster that does not have a one-to-one correspondence between the network and disk modules. That is, different nodes can have a different number of network and disk modules, and the same node can have a different number of network modules than disk modules.
(41) Further, a host device **108**, **110** can be networked with the nodes **116**, **118** in the cluster, over the storage networking connections **112**, **114**. As an example, respective host devices **108**, **110** that are networked to a cluster may request services (e.g., exchanging of information in the form of data packets) of nodes **116**, **118** in the cluster, and the nodes **116**, **118** can return results of the requested services to the host devices **108**, **110**. In an embodiment, the host devices **108**, **110** can exchange information with the network modules **120**, **122** residing in the nodes **116**, **118** (e.g., network hosts) in the data storage systems **102**, **104**.
(42) In an embodiment, the data storage devices **128**, **130** comprise volumes **132**, which is an implementation of storage of information onto disk drives or disk arrays or other storage (e.g., flash) as a file-system for data, for example. In an example, a disk array can include all traditional hard drives, all flash drives, or a combination of traditional hard drives and flash drives. Volumes can span a portion of a disk, a collection of disks, or portions of disks, for example, and typically define an overall logical arrangement of file storage on disk space in the storage system. In an embodiment a volume can comprise stored data as one or more files that reside in a hierarchical directory structure within the volume.
(43) Volumes are typically configured in formats that may be associated with particular storage systems, and respective volume formats typically comprise features that provide functionality to the volumes, such as providing an ability for volumes to form clusters. For example, where a first storage system may utilize a first format for their volumes, a second storage system may utilize a second format for their volumes.
(44) In the clustered network environment **100**, the host devices **108**, **110** can utilize the data storage systems **102**, **104** to store and retrieve data from the volumes **132**. In this embodiment, for example, the host device **108** can send data packets to the network module **120** in the node **116** within data storage system **102**. The node **116** can forward the data to the data storage device **128**

using the disk module **124**, where the data storage device **128** comprises volume **132**A. In this way, in this example, the host device can access the volume **132**A, to store and/or retrieve data, using the data storage system **102** connected by the storage network connection **112**. Further, in this embodiment, the host device **110** can exchange data with the network module **122** in the node **118** within the data storage system **104** (e.g., which may be remote from the data storage system **102**). The node **118** can forward the data to the data storage device **130** using the disk module **126**, thereby accessing volume **1328** associated with the data storage device **130**.

(45) It may be appreciated that managing objects within an object store using an object file system may be implemented within the clustered network environment **100**, such as where nodes within the clustered network environment store data as objects within a remote object store. It may be appreciated that managing objects within an object store using an object file system may be implemented for and/or between any type of computing environment, and may be transferrable between physical devices (e.g., node **116**, node **118**, a desktop computer, a tablet, a laptop, a wearable device, a mobile device, a storage device, a server, etc.) and/or a cloud computing environment (e.g., remote to the clustered network environment **100**).

(46) FIG. **2** is an illustrative example of a data storage system **200** (e.g., **102**, **104** in FIG. **1**), providing further detail of an embodiment of components that may implement one or more of the techniques and/or systems described herein. The data storage system **200** comprises a node **202** (e.g., nodes **116**, **118** in FIG. **1**), and a data storage device **234** (e.g., data storage devices **128**, **130** in FIG. **1**). The node **202** may be a general purpose computer, for example, or some other computing device particularly configured to operate as a storage server. A host device **205** (e.g., **108**, **110** in FIG. **1**) can be connected to the node **202** over a network **216**, for example, to provide access to files and/or other data stored on the data storage device **234**. In an example, the node **202** comprises a storage controller that provides client devices, such as the host device **205**, with access to data stored within data storage device **234**.

(47) The data storage device **234** can comprise mass storage devices, such as disks **224**, **226**, **228** of a disk array **218**, **220**, **222**. It will be appreciated that the techniques and systems, described herein, are not limited by the example embodiment. For example, disks **224**, **226**, **228** may comprise any type of mass storage devices, including but not limited to magnetic disk drives, flash memory, and any other similar media adapted to store information, including, for example, data (D) and/or parity (P) information.

(48) The node **202** comprises one or more processors **204**, a memory **206**, a network adapter **210**, a cluster access adapter **212**, and a storage adapter **214** interconnected by a system bus **242**. The data storage system **200** also includes an operating system **208** installed in the memory **206** of the node **202** that can, for example, implement a Redundant Array of Independent (or Inexpensive) Disks (RAID) optimization technique to optimize a reconstruction process of data of a failed disk in an array.

(49) The operating system **208** can also manage communications for the data storage system, and communications between other data storage systems that may be in a clustered network, such as attached to a cluster fabric **215** (e.g., **106** in FIG. **1**). Thus, the node **202**, such as a network storage controller, can respond to host device requests to manage data on the data storage device **234** (e.g., or additional clustered devices) in accordance with these host device requests. The operating system **208** can often establish one or more file systems on the data storage system **200**, where a file system can include software code and data structures that implement a persistent hierarchical namespace of files and directories, for example. As an example, when a new data storage device (not shown) is added to a clustered network system, the operating system **208** is informed where, in an existing directory tree, new files associated with the new data storage device are to be stored. This is often referred to as “mounting” a file system.

(50) In the example data storage system **200**, memory **206** can include storage locations that are addressable by the processors **204** and adapters **210**, **212**, **214** for storing related software

application code and data structures. The processors **204** and adapters **210**, **212**, **214** may, for example, include processing elements and/or logic circuitry configured to execute the software code and manipulate the data structures. The operating system **208**, portions of which are typically resident in the memory **206** and executed by the processing elements, functionally organizes the storage system by, among other things, invoking storage operations in support of a file service implemented by the storage system. It will be apparent to those skilled in the art that other processing and memory mechanisms, including various computer readable media, may be used for storing and/or executing application instructions pertaining to the techniques described herein. For example, the operating system can also utilize one or more control files (not shown) to aid in the provisioning of virtual machines.

(51) The network adapter **210** includes the mechanical, electrical and signaling circuitry needed to connect the data storage system **200** to a host device **205** over a network **216**, which may comprise, among other things, a point-to-point connection or a shared medium, such as a local area network. The host device **205** (e.g., **108**, **110** of FIG. **1**) may be a general-purpose computer configured to execute applications. As described above, the host device **205** may interact with the data storage system **200** in accordance with a client/host model of information delivery.

(52) The storage adapter **214** cooperates with the operating system **208** executing on the node **202** to access information requested by the host device **205** (e.g., access data on a storage device managed by a network storage controller). The information may be stored on any type of attached array of writeable media such as magnetic disk drives, flash memory, and/or any other similar media adapted to store information. In the example data storage system **200**, the information can be stored in data blocks on the disks **224**, **226**, **228**. The storage adapter **214** can include input/output (I/O) interface circuitry that couples to the disks over an I/O interconnect arrangement, such as a storage area network (SAN) protocol (e.g., Small Computer System Interface (SCSI), iSCSI, hyperSCSI, Fiber Channel Protocol (FCP)). The information is retrieved by the storage adapter **214** and, if necessary, processed by the one or more processors **204** (or the storage adapter **214** itself) prior to being forwarded over the system bus **242** to the network adapter **210** (and/or the cluster access adapter **212** if sending to another node in the cluster) where the information is formatted into a data packet and returned to the host device **205** over the network **216** (and/or returned to another node attached to the cluster over the cluster fabric **215**).

(53) In an embodiment, storage of information on disk arrays **218**, **220**, **222** can be implemented as one or more storage volumes **230**, **232** that are comprised of a cluster of disks **224**, **226**, **228** defining an overall logical arrangement of disk space. The disks **224**, **226**, **228** that comprise one or more volumes are typically organized as one or more groups of RAIDs. As an example, volume **230** comprises an aggregate of disk arrays **218** and **220**, which comprise the cluster of disks **224** and **226**.

(54) In an embodiment, to facilitate access to disks **224**, **226**, **228**, the operating system **208** may implement a file system (e.g., write anywhere file system) that logically organizes the information as a hierarchical structure of directories and files on the disks. In this embodiment, respective files may be implemented as a set of disk blocks configured to store information, whereas directories may be implemented as specially formatted files in which information about other files and directories are stored.

(55) Whatever the underlying physical configuration within this data storage system **200**, data can be stored as files within physical and/or virtual volumes, which can be associated with respective volume identifiers, such as file system identifiers (FSIDs), which can be 32-bits in length in one example.

(56) A physical volume corresponds to at least a portion of physical storage devices whose address, addressable space, location, etc. doesn't change, such as at least some of one or more data storage devices **234** (e.g., a Redundant Array of Independent (or Inexpensive) Disks (RAID system)). Typically the location of the physical volume doesn't change in that the (range of) address(es) used

to access it generally remains constant.
(57) A virtual volume, in contrast, is stored over an aggregate of disparate portions of different physical storage devices. The virtual volume may be a collection of different available portions of different physical storage device locations, such as some available space from each of the disks **224**, **226**, and/or **228**. It will be appreciated that since a virtual volume is not "tied" to any one particular storage device, a virtual volume can be said to include a layer of abstraction or virtualization, which allows it to be resized and/or flexible in some regards.
(58) Further, a virtual volume can include one or more logical unit numbers (LUNs) **238**, directories **236**, Qtrees **235**, and files **240**. Among other things, these features, but more particularly LUNS, allow the disparate memory locations within which data is stored to be identified, for example, and grouped as data storage unit. As such, the LUNs **238** may be characterized as constituting a virtual disk or drive upon which data within the virtual volume is stored within the aggregate. For example, LUNs are often referred to as virtual drives, such that they emulate a hard drive from a general purpose computer, while they actually comprise data blocks stored in various parts of a volume.
(59) In an embodiment, one or more data storage devices **234** can have one or more physical ports, wherein each physical port can be assigned a target address (e.g., SCSI target address). To represent respective volumes stored on a data storage device, a target address on the data storage device can be used to identify one or more LUNs **238**. Thus, for example, when the node **202** connects to a volume **230**, **232** through the storage adapter **214**, a connection between the node **202** and the one or more LUNs **238** underlying the volume is created.
(60) In an embodiment, respective target addresses can identify multiple LUNs, such that a target address can represent multiple volumes. The I/O interface, which can be implemented as circuitry and/or software in the storage adapter **214** or as executable code residing in memory **206** and executed by the processors **204**, for example, can connect to volume **230** by using one or more addresses that identify the one or more LUNs **238**.
(61) It may be appreciated that managing objects within an object store using an object file system may be implemented for the data storage system **200**. It may be appreciated that managing objects within an object store using an object file system may be implemented for and/or between any type of computing environment, and may be transferrable between physical devices (e.g., node **202**, host device **205**, a desktop computer, a tablet, a laptop, a wearable device, a mobile device, a storage device, a server, etc.) and/or a cloud computing environment (e.g., remote to the node **202** and/or the host device **205**).
(62) One embodiment of managing objects within an object store using an object file system is illustrated by an exemplary method **300** of FIG. **3** and further described in conjunction with system **400** of FIG. **4**A. A computing device **402** may comprise a node, a storage controller, a storage service, an on-premises computing device, a storage virtual machine, or any other hardware or software. The computing device **402** may store data **406** within storage devices (primary storage) managed by the computing device **402**. The computing device **402** may provide client devices with access to the data **406**, such as by processing read and write operations from the client devices. The computing device **402** may create snapshots **404** of the data **406**, such as a snapshot of a file system of a volume accessible to the client devices through the computing device **402**. The computing device **402** may be configured to communicate with an object store **409** over a network. The object store **409** may comprise a cloud computing environment remote to the computing device **402**.
(63) As provided herein, an object file system and object format is provided for storing and accessing data, such as snapshots, stored within objects in the object store **409**. At **302**, the data **406**, maintained by the computing device, is stored into a plurality of slots of an object **408**. Each slot represents a base unit of data of the object file system defined for the object store **409**. For example, the object **408** comprises 1024 or any other number of slots, wherein each slot comprises 4 kb of data or any other amount of data. It may be appreciated that objects may comprise any

number of slots of any size. User data, directory blocks, metadata, and/or inofile blocks of an inofile comprising per inode metadata is stored into the slots of the object **408**. In an example, snapshot data, of a snapshot created by the computing device **402** of a file system maintained by the computing device **402**, is stored into the object **408**. For example, the object **408** may be maintained as an independent logical representation of the snapshot, such that data of the snapshot is accessible through the object **408** without having to reference other logical copies of other snapshots stored within objects **410** of the object store **409**. In an example, the data is converted from physical data into a version independent format for storage within the object **408**.

(64) In an example, the object **408** is created to comprise data in a compressed state corresponding to compression of the data within the primary storage of the computing device **402**. In this way, compression used by the computing device **402** to store the data is retained within the object **408** for storage within the object store **409**. The object **408** may be assigned a unique sequence number. Each object within the object store **409** is assigned unique sequence numbers.

(65) An object header may be created for the object **408**. The object header comprises a slot context for slots within the object **408**. The slot context may comprise information relating to a type of compression used for compressing data within the object **408** (if any compression is used), a start offset of a slot, a logical data length, a compressed data length, etc. The slot context may be used to access compressed data stored within the object **408**.

(66) FIG. **4**C illustrates an example of the object **408**. The object **408** comprises an object header **436** and a plurality of slots, such as a slot **426**, a second **428**, a slot **430**, and/or any other number of slots. The object header **436** may have a size that is aligned with a start of the plurality of slots, such as having a 4 kb alignment based upon each slot having a logical length of 4 kb. It may be appreciated that slots may have any length. The object header **436** comprises various information, such as a version identifier, a header checksum, a length of the object **408**, a slot context **432**, and/or other information used to access and manage data populated into the slots of the object **408**.

(67) The slot context **432** comprises various information about the slots, such as a compression type of a slot (e.g., a type of compression used to compress data of slots into a compression group or an indicator that the slot does not comprise compressed data), a start offset of the slot within the object **408** (e.g., a slot identifier multiplied by a slot size, such as 4 kb), a logical data length of the slot (e.g., 4 kb), a compressed length (e.g., 0 if uncompressed), an index of the slot within a compression group of multiple slots (e.g., 0 if uncompressed), a logical data checksum, etc.

(68) At **304**, the data stored within the slots of the object **408** are represented as a data structure. The data structure may comprise a tree structure or any other type of structure. For example, the data structure comprises the tree structure representing a file. The data structure may be populated with a plurality of nodes at various levels of the tree structure. The nodes may be represented by cloud block numbers. A cloud block number of a node may comprise a sequence number used to uniquely identify the object **408** and/or a slot number of a slot comprising a portion of the data represented by the node. User data, directory blocks, metadata, inofile blocks of an inofile, and/or other data stored within the slots of the object **408** may be represented by nodes within the data structure. In an example, user data is stored within leaf nodes of the data structure (e.g., nodes within a level 0 (L0) level of the tree structure). Pointers (indirects) may be stored within non-leaf nodes of the data structure (e.g., nodes within a level 1(L1), a level 2 (L2), and/or other levels of the tree structure). An inode object for the file may comprise pointers that point to non-leaf nodes within a top level of the data structure.

(69) In an example of the tree structure, a 1 TB file may be represented by the tree structure. An inode of the file may comprise metadata and/or a flat list of 3845 pointers or any other number of pointers to nodes within a level 2 of the tree structure (e.g., there are 3845 nodes (4 kb blocks) within the level 2 of the tree structure). The level 2 comprises the 3845 nodes (4 kb blocks), each having 255 pointers or any other number of pointers to nodes within a level 1 of the tree structure (e.g., there are 980393 (4 kb blocks) within the level 1 of the tree structure. The level 1 comprises

the 980393 (4 kb blocks), each having 255 pointers to nodes within a level 0 of the tree structure. The level 0 comprises 250,000,000 nodes (4 kb blocks) representing actual data, such as user data.

(70) FIG. **4**B illustrates a snapshot file system of data structures **424** used to represent snapshots (e.g., snapshots of one or more volumes managed by the computing device **402**) stored into the objects **410** of the object store **409**. There is one base root object per volume, such as a base root object **412** for a volume of which the snapshots were captured. There is a unique root object per volume, such as a unique root object **414** for the volume. The base root object **412** may point to the unique root object **414**. Names of the unique root objects may be derived from increasing generation numbers. The unique root object **414** may point to snapinfo objects, such as a snapinfo object **416** comprising information regarding one or more snapshots, such as a pointer to an inofile **418** of a second snapshot of the volume. The inofile **418** comprises cloud block numbers of slots within an object comprising data of the second snapshot, such as a pointer to an indirect **420** that points to data **422** of the snapshot. The inofile **418** may comprise or point to information relating to directories, access control lists, and/or other information.

(71) At **306**, a mapping metafile (a VMAP) is maintained for the object **408**. The mapping metafile maps block numbers of primary storage of the computing device **402** (e.g., virtual volume block numbers of the data stored into slots of the object **408**) to cloud block numbers of nodes representing portions of the data stored within the slots of the object **408**. At **308**, the object **408** is stored within the object store **409**. In an example of storing objects into the object store **409**, the plurality of snapshots **404**, maintained by the computing device **402**, are stored within objects **410** of the object store **409**. Each snapshot is identifiable through a snapinfo object that has a unique generation number. As will be described later, the objects **410** within the object store **409** may be deduplicated with respect to one another (e.g., the object **408** is deduplicated with respect to the object **410** using the mapping metafile as part of being stored into the object store **409**) and retain compression used by the computing device **402** for storing the snapshots **404** within the primary storage.

(72) At **310**, the mapping metafile and/or the data structure are used to provide access through the object file system to portions of data within the slots of the object **408** in the object store **409**. In an example, the inode object and the data structure are traversed to identify a sequence number and slot number of requested data. The sequence number and the slot number are used to access the requested data within a corresponding slot of the object **408**. In an example, a read request targets a 100,000th level 0 block stored within the object **408**. The inode object is read to calculate which blocks in each level of the data structure will have 100,000 (e.g., 100,000/255 is a 393th block in level 1 and 393/255 is a 2.sup.nd block in level 2). These blocks are read at each level to go to a next level through appropriate pointers (e.g., cloud block numbers) until the data is read from a block of user data within the level 0. The pointers are cloud block numbers, where a pointer comprises a sequence number of the object **408** and a slot number. The sequence number corresponds to an object name of the object **408** and the slot number is which slot the data is located within the object **408**.

(73) In an embodiment, an on-demand restore of data within a snapshot stored within objects of the object store **409** can be performed to a target computing device using the mapping metafile and/or the data structure. In an embodiment, the mapping metafile and/or the data structure may be used to free objects from the object store **409** based upon the objects comprising snapshot data of snapshots deleted by the computing device **402**.

(74) In an embodiment, the mapping metafile and/or an overflow mapping metafile are used to facilitate the copying of the snapshots to the object store **409** in a manner that preserves deduplication and compression, logically represents the snapshots as fully independent snapshots, and provides additional compression. In particular, the mapping metafile is populated with entries for block numbers (e.g., virtual volume block numbers, physical volume block numbers, etc. used by the node to reference data such as snapshot data stored by the node) of the snapshots **404**

maintained by the computing device **402** and copied into the objects **410** of the object store **409** as copied snapshots. An entry within the mapping metafile is populated with a mapping between a block number of data within a snapshot at the computing device **402** (e.g., a virtual volume block number) and a cloud block number (e.g., a cloud physical volume block number) of a slot within an object into which the data was copied when the snapshot was copied to the object store **409** as a copied snapshot. The entry is populated with a compression indicator to indicate whether data of the block number is compressed or not (e.g., a bit set to a first value to indicate a compressed virtual volume block number and set to a second value to indicate a non-compressed virtual volume block number).

(75) The entry is populated with a compression group start indicator to indicate whether the block number is a starting block number for a compression group of a plurality of block numbers of compressed data blocks. The entry is populated with an overflow indicator to indicate whether the data block has an overflow entry within the overflow mapping metafile. The overflow mapping metafile may comprise a V+ tree, such as a special B+ tree with support for variable length key and payload so a key can be sized according to a type of entry being stored for optimization. The key uniquely represents all types of entries associated with a block number (a virtual volume block number). The key may comprise a block number field (e.g., the virtual volume block number of a data block represented by the block number or a starting virtual volume block number of a first data block of a compression group comprising the data block), a physical length of an extent of the data block, if the corresponding entry is a start of a compression group, and other block numbers of blocks within the compression group. The payload is a cloud block number (a cloud physical volume block number). The entry may be populated with a logical length of an extent associated with the block number. The entry may be populated with a physical length of the extent associated with the block number.

(76) The mapping metafile and/or the overflow mapping metafile may be indexed by block numbers of the primary storage (e.g., virtual volume block numbers of snapshots stored by the computing device **402** within the primary storage, which are copied to the object store as copied snapshots). In an example, the block numbers may correspond to virtual volume block numbers of data of the snapshots stored by the computing device **402** within the primary storage. In an example, a block number corresponds to a starting virtual volume block number of an extent of a compression group.

(77) The mapping metafile and/or the overflow mapping metafile is maintained according to a first rule specifying that the mapping metafile and/or the overflow mapping metafile represent a comprehensive set of cloud block numbers corresponding to a latest snapshot copied to the object. The mapping metafile and/or the overflow mapping metafile is maintained according to a second rule specifying that entries within the mapping metafile and/or the overflow mapping metafile are invalidated based upon any block number in the entries being freed by the computing device **402**.

(78) The mapping metafile and/or the overflow mapping metafile is used to determine what data of the current snapshot is to be copied to the object store **409** and what data already exists within the object store **409** so that only data not already within the object store **409** is transmitted to the object store **409** for storage within an object. Upon determining that the current snapshot is to be copied to the object store **409**, an invalidation phase is performed. In particular, a list of deallocated block numbers of primary storage of the computing device **402** (e.g., virtual volume block numbers, of the file system of which snapshots are created, that are no longer being actively used to store in-use data by the node) are determined based upon a difference between a first snapshot and a second snapshot of the primary storage (e.g., a difference between a base snapshot and an incremental snapshot of the file system). As part of the invalidation phase, entries for the list of deallocated block numbers are removed from the mapping metafile and/or the overflow mapping metafile.

(79) After the invalidation phase, a list of changed block numbers corresponding to changes between the current snapshot of the primary storage being copied to the object store **409** and a prior

copied snapshot already copied from the primary storage to the object store **409** is determined. The mapping metafile is evaluated using the list of changed block numbers to identify a deduplicated set of changed block numbers without entries within the mapping metafile. The deduplicated set of changed block numbers correspond to data, of the current snapshot, not yet stored within the object store **409**.

(80) An object is created to store data of the deduplicated set of changed block numbers. The object comprises a plurality of slots, such as 1024 or any other number of slots. The data of the deduplicated set of changed block numbers is stored into the slots of the object. An object header is updated with metadata describing the slots. In an example, the object is created to comprise the data in a compressed state corresponding to compression of the data in the primary storage. The object can be compressed by combining data within contiguous slots of the object into a single compression group. In this way, compression of the current snapshot maintained by the node is preserved when the current snapshot is stored in the object store as the object corresponding to a copy of the current snapshot.

(81) The object, comprising the data of the deduplicated set of changed block numbers, is transmitted to the object store **409** for storage as a new copied snapshot that is a copy of the current snapshot maintained by the node. The object is stored as a logical copy of the current snapshot. Also, additional compression is applied to this logical data, and information used to uncompress the logical data is stored in the object header. Further, the object is maintained as an independent logical representation of the current snapshot, such that copied data, copied from the current snapshot, is accessible through the object without having to reference other logical copies of other copied snapshots stored in other objects within the object store **409**. Once the object is stored within the object store **409**, the mapping metafile and/or the overflow mapping metafile is updated with entries for the deduplicated set of changed block numbers based upon receiving an acknowledgment of the object being stored by the object store **409**. An entry will map a changed block number to a cloud block number of a slot within which data of the changed block number is stored in the object.

(82) In an embodiment, the object file system is used to provide various primary storage system services for the object store **409** in order to achieve efficient space and resource management, and flexible scaling in the object store **409** (e.g., a cloud computing environment). Additionally, pseudo read only snapshots are provided through the object store **409**. Consumers of these snapshots may choose to derive just the logical data represented by these snapshots or can additionally derive additional metadata associated with the logical data if required. This additional metadata is created post snapshot creation and hence is not directly part of logical view of the snapshot. The present system provides flexible, scalable, and cost effective techniques for leveraging cloud storage for off-premises operations on secondary data, such as analytics, development testing, virus scan, load distribution, etc. Objects may be modified (e.g., a unit of storage within a cloud storage environment) without changing the meaning or accessibility of useable data in the objects (e.g., a cloud object comprising a snapshot copy of primary data maintained by the computing device **402**). Objects may be modified to add additional metadata and information such as analytics data, virus scan data, etc. to useable data without modifying the useable data. Thus, an object is maintained as a pseudo read only object because in-use data is unmodifiable while unused or freed data is modifiable such as by a defragmentation and/or garbage collection process.

(83) Changes in objects can be detected in order to resolve what data of the objects is the correct data. The present system provides the ability to perform defragmentation and garbage collection for objects by a cloud service hosted by the object store **409**, such as a cloud storage environment. Defragmentation and garbage collection are provided without affecting access to other in-use data within objects (e.g., in-use snapshot data stored within an object that is used by one or more applications at various remote computers). This allows for more true distributed and infinite scale data management. The present system provides for the ability to run analytics on objects (e.g.,

read/write analytics of data access to data within an object) using analytic applications hosted within the cloud storage environment. The analytics can be attached to objects even though the objects are read only. The present system provides for deduplication of objects. In this way, objects can be modified while still maintaining consistency of in-use data within the objects (e.g., maintaining consistency of a file system captured by a snapshot that is stored within an object) and without compromising a read only attribute of the objects. Also, computationally expensive processes like garbage collection, analytics, and defragmentation are offloaded from on-premises primary storage systems, such as the computing device **402**, to the object store **409** such as cloud services within the cloud storage environment.

(84) In one embodiment, objects within the object store **409** (e.g., objects within a cloud computing environment) can be maintained with a read only attribute such that data within objects can be overwritten/modified/freed so long as in-use data within the objects is not altered. In particular, an object may be maintained within the object store **409**, such as a cloud computing environment. The object comprises a plurality of slots, such as 1024 or any other number of slots. Each slot is used to store a unit of data. The data within each slot is read-only. In particular, the data is read only when in-use, such as where one or more applications are referencing or using the data (e.g., an application hosted by the computing device **402** is storing data of a snapshot of a local file system within a slot of an object, and thus the snapshot data is in-use until a particular event occurs such as the computing device **402** deleting the snapshot). In an example, the object comprises snapshot data of a file system, a volume, a logical unit number (LUN), a file, or any other data of the computing device **402**. In this way, the object comprises a read only snapshot of data of the computing device **402**. In one example, a plurality of objects corresponding to read only snapshots of the file system of the computing device **402** are stored within the object store **409**. Each object is assigned a unique sequence identifier.

(85) A first rule is enforced for the object. The first rule specifies that in-use slots are non-modifiable and unused slots are modifiable. An in-use slot is a slot that stores data actively referenced, used, and/or maintained by a computing device **402** (a primary storage system). For example, an in-use slot may be a slot that comprises snapshot data (e.g., secondary/replicated data) of a snapshot created by a computing device **402**. The slot becomes an unused slot when the data is no longer actively referenced, used, and/or maintained, such as where the computing device **402** deletes the snapshot. Thus, if a slot is in-use, then the data within the slot cannot be modified. Otherwise, data in unused slots (e.g., stale data that is no longer referenced or used) can be modified, such as deleted/freed by garbage collection functionality or defragmentation functionality.

(86) Additional information for the object may be generated. The additional information may comprise analytics (e.g., read/write statistics of access to the object), virus scan information, development testing data, and/or a variety of other information that can be generated for the object and the data stored therein. In an example, the additional data is generated by a cloud service or application executing within the cloud computing environment. This will offload processing and resource utilization that would otherwise be used by the computing device **402** (primary storage system) to perform such analytics and processing.

(87) Metadata of the additional information is attached to an object header of the object. The object header is used to store metadata for each slot of the object. In one example, the metadata specifies a location of the additional information within the object, such as a particular slot into which the additional information is stored. In another example, the metadata may comprise the additional information, and thus the additional information is stored into the object header. The metadata is attached in a manner that does not change a meaning or accessibility of useable data within in-use slots of the object. In particular, applications that are allowed to merely access user data within the object (e.g., the applications are unaware or have no reason to access the additional information) are provided with only access to the user data and are not provided with access to the metadata or

additional information. Thus, these applications continue to access user data within the object in a normal manner. For application that are allowed to access both the user data and the additional information, those applications are provided with access to the user data and the metadata for identifying and accessing a location of the additional information within the object. The first rule is enforced such that user data (in-use data) is retained in an unmodified state within the object notwithstanding the metadata and/or additional information being associated with the object.

(88) In an example, a second rule is enforced for the object. The second rule specifies that related read operations are to be directed to a same version of an object. For example, an object corresponds to secondary/replicated snapshot data of a file system maintained by the computing device **402**. Each time a new snapshot of the file system is created, a new version of the object is created to capture changes to the file system. In another example, since in-use data within the object is read only and unmodifiable, any modifications to slots with in-use data will result in a new version of the object being created with the modified data.

(89) If multiple read operations are related, then those read operations should be executed upon the same version of the object for data consistency purposes. This is achieved by comparing timestamp data of the related read operations. If the timestamp data between the related read operations is mismatched, then the related read operations are retried because the related read operations were executed upon different versions of the same object. If the timestamp data between the read operations matches, then the related read operations are considered successful. In an example, a first related read operation reads the object header of the object to identify a slot from which data is to be read. A second related read operation is executed to read data from the slot. The two related read operations should be executed upon the same version of the object/slot (e.g., the operations can be executed upon different versions such as where data of a current version of the object is modified between execution of the operations, thus creating a new version of the object with the modified data since the object is read only and the original data is unmodifiable within the current version of the object). Thus, timestamp data of the two related read operations is used to determine whether the two related read operations were executed upon the same version of the object/slot and thus should be considered complete or should be retried.

(90) In one embodiment, garbage collection is provided for objects within the object store **409**. The objects have a read only state, such that enforcement of the first rule ensures that in-use data within slots of an object is not modifiable, thus making objects pseudo read only objects because only unused slots can be modified/freed of unused data. In an example, an object is used to store data of a snapshot of a file system hosted by the computing device **402**. The snapshot may be determined as being deleted by the computing device **402**, and thus slots comprising snapshot data of the deleted snapshot are now considered to be unused slots as opposed to in-use slots.

(91) Each snapshot of the file system may be associated with a bitmap that identifies objects within the object store that correspond to a particular snapshot. Thus, the bitmaps can be evaluated to identify what objects comprise data of particular snapshots. For example, a bitmap of the deleted snapshot can be used to identify the object and other objects as comprising data of the deleted snapshot.

(92) A garbage collection operation is executed to free objects (e.g. free unused data from unused slots) from the object store in order to reduce storage utilization of the object store that would otherwise be unnecessarily used to store stale/unused data. In an example, the garbage collection operation is executed by a cloud service in order to conserve resource consumption by the computing device **402** (primary storage system) otherwise used to execute the garbage collection operation. The garbage collection operation free objects from the object store **409** based upon the objects uniquely corresponding to deleted snapshots. That is, if an object stores data of only deleted snapshots and does not store data of active/undeleted snapshots, then the garbage collection process can free/delete that object. For example, the bitmaps describing objects within the object store **409** that are related to snapshots of the file system are evaluated to determine whether the object is

unique to the deleted snapshot and/or unique to only deleted snapshots (e.g., the object does not comprise data of active/undeleted snapshots). If so, then the object is freed from the object store **409**. However, if the object is not unique to only deleted snapshot(s) such as where the object also stores data of an active/undeleted snapshot, then the object is not freed.

(93) In an embodiment, defragmentation is provided for fragmented objects within the object store **409**. In an example, defragmentation is implemented by a cloud service or application executing in the object store **409** in order to conserve resources otherwise used by a computing device **402** (primary storage system) that would execute defragmentation functionality. An object within the object store **409** is determined to be a fragmented object based upon the object comprising at least one freed slot from which data was freed. For example, a freed slot may comprise an unused slot comprising unused data no longer referenced/used by the computing device **402** (e.g., data of a deleted snapshot). Accordingly, the fragmented object may comprise one or more in-use slots of in-use data currently referenced/used by a computing device **402** and one or more freed slots of freed data (e.g., unused slots comprising unused data).

(94) The fragmented object is compacted to retain the in-use data and exclude the freed data (the unused data) as a written object. Because compacting may store the in-use data in new slots, an object header of the object is updated with new locations of the in-use data within the rewritten object. In this way, defragmentation is performed for objects within the object store **409**.

(95) The present system preserves deduplication and compression used by the computing device **402** for snapshots when storing copied snapshots to the object store **409** notwithstanding copied snapshots representing fully logical copies of data in the primary storage of the computing device **402**. In particular, deduplication is preserved because data that is shared in a snapshot (e.g., a local or primary snapshot created and maintain by the node) is also shared in a copied snapshot in the object store **409**. Deduplication of compression groups is maintained while logically representing the compression groups in a copied snapshot. Block sharing across multiple snapshots is also preserved so that merely changed blocks are transferred/copied to the object store **490** during incremental snapshot transfers.

(96) The present system provides additional compression on a snapshot data copy. In particular, larger compression groups provide more space efficiency but with less read efficiency compared to smaller compression groups. Relatively smaller compression groups may be used by the computing device **402** of the storage system since access to the primary storage of the computing device **402** may be more read intensive, and thus read efficiency is prioritized over storage space efficiency. Because copied snapshots in the object store **409** are infrequently accessed (e.g., cold data that is infrequently read), relatively larger compression groups can be employed for improved storage space efficiency within the object store, which also reduces network bandwidth for snapshot copying to the object store **409**.

(97) In one embodiment, snapshots maintained by the computing device **402** are copied to the object store **409** as copied snapshots representing logical data of the snapshots. Data of the copied snapshots is stored into slots of objects that are deduplicated with respect to other objects stored within the object store **409** and retain compression used by the computing device **402** for the snapshots.

(98) In an example, the computing device **402** stores data within primary storage. The computing device **402** may create snapshots of the data stored by the computing device **402**. For example, the computing device **402** may create a snapshot of a file, a logical unit number, a directory, a volume, a storage virtual machine hosting a plurality of volumes, a file system, a consistency group of any arbitrary grouping of files, directories, or data, etc. The computing device **402** may deduplicate data between the snapshots so that instead of storing redundant data blocks multiple times, merely references are stored in place of the redundant data blocks and point to original data blocks with the same data. The computing device **402** may compress data within the snapshots, such as by creating compression groups of compressed data blocks.

(99) The mapping metafile and/or the overflow mapping metafile is used to determine what data of the current snapshot is to be copied to the object store **409** and what data already exists within the object store so that only data not already within the object store is transmitted to the object store **409** for storage within an object. Upon determining that the current snapshot is to be copied to the object store, an invalidation phase is performed. In particular, a list of deallocated block numbers of primary storage of the computing device **402** (e.g., virtual volume block numbers, of the file system of which snapshots are created, that are no longer being actively used to store in-use data by the node) are determined based upon a difference between a first snapshot and a second snapshot of the primary storage (e.g., a difference between a base snapshot and an incremental snapshot of the file system). As part of the invalidation phase, entries for the list of deallocated block numbers are removed from the mapping metafile and/or the overflow mapping metafile.

(100) Still another embodiment involves a computer-readable medium **500** comprising processor-executable instructions configured to implement one or more of the techniques presented herein. An example embodiment of a computer-readable medium or a computer-readable device that is devised in these ways is illustrated in FIG. **5**, wherein the implementation comprises a computer-readable medium **508**, such as a compact disc-recordable (CD-R), a digital versatile disc-recordable (DVD-R), flash drive, a platter of a hard disk drive, etc., on which is encoded computer-readable data **506**. This computer-readable data **506**, such as binary data comprising at least one of a zero or a one, in turn comprises a processor-executable computer instructions **504** configured to operate according to one or more of the principles set forth herein. In some embodiments, the processor-executable computer instructions **504** are configured to perform a method **502**, such as at least some of the exemplary method **300** of FIG. **3**, for example. In some embodiments, the processor-executable computer instructions **504** are configured to implement a system, such as at least some of the exemplary system **400** of FIG. **4**A, for example. Many such computer-readable media are contemplated to operate in accordance with the techniques presented herein.

(101) FIG. **6** is a diagram illustrating an example operating environment **600** in which an embodiment of the techniques described herein may be implemented. In one example, the techniques described herein may be implemented within a client device **628**, such as a laptop, tablet, personal computer, mobile device, wearable device, etc. In another example, the techniques described herein may be implemented within a storage controller **630**, such as a node configured to manage the storage and access to data on behalf of the client device **628** and/or other client devices. In another example, the techniques described herein may be implemented within a distributed computing platform **602** such as a cloud computing environment (e.g., a cloud storage environment, a multi-tenant platform, etc.) configured to manage the storage and access to data on behalf of the client device **628** and/or other client devices.

(102) In yet another example, at least some of the techniques described herein are implemented across one or more of the client device **628**, the storage controller **630**, and the distributed computing platform **602**. For example, the client device **628** may transmit operations, such as data operations to read data and write data and metadata operations (e.g., a create file operation, a rename directory operation, a resize operation, a set attribute operation, etc.), over a network **626** to the storage controller **630** for implementation by the storage controller **630** upon storage. The storage controller **630** may store data associated with the operations within volumes or other datan objects/structures hosted within locally attached storage, remote storage hosted by other computing devices accessible over the network **626**, storage provided by the distributed computing platform **602**, etc. The storage controller **630** may replicate the data and/or the operations to other computing devices so that one or more replicas, such as a destination storage volume that is maintained as a replica of a source storage volume, are maintained. Such replicas can be used for disaster recovery and failover.

(103) The storage controller **630** may store the data or a portion thereof within storage hosted by the distributed computing platform **602** by transmitting the data to the distributed computing

platform **602**. In one example, the storage controller **630** may locally store frequently accessed data within locally attached storage. Less frequently accessed data may be transmitted to the distributed computing platform **602** for storage within a data storage tier **608**. The data storage tier **608** may store data within a service data store **620**, and may store client specific data within client data stores assigned to such clients such as a client (1) data store **622** used to store data of a client (1) and a client (N) data store **624** used to store data of a client (N). The data stores may be physical storage devices or may be defined as logical storage, such as a virtual volume, LUNs, or other logical organizations of data that can be defined across one or more physical storage devices. In another example, the storage controller **630** transmits and stores all client data to the distributed computing platform **602**. In yet another example, the client device **628** transmits and stores the data directly to the distributed computing platform **602** without the use of the storage controller **630**.

(104) The management of storage and access to data can be performed by one or more storage virtual machines (SMVs) or other storage applications that provide software as a service (SaaS) such as storage software services. In one example, an SVM may be hosted within the client device **628**, within the storage controller **630**, or within the distributed computing platform **602** such as by the application server tier **606**. In another example, one or more SVMs may be hosted across one or more of the client device **628**, the storage controller **630**, and the distributed computing platform **602**.

(105) In one example of the distributed computing platform **602**, one or more SVMs may be hosted by the application server tier **606**. For example, a server (1) **616** is configured to host SVMs used to execute applications such as storage applications that manage the storage of data of the client (1) within the client (1) data store **622**. Thus, an SVM executing on the server (1) **616** may receive data and/or operations from the client device **628** and/or the storage controller **630** over the network **626**. The SVM executes a storage application to process the operations and/or store the data within the client (1) data store **622**. The SVM may transmit a response back to the client device **628** and/or the storage controller **630** over the network **626**, such as a success message or an error message. In this way, the application server tier **606** may host SVMs, services, and/or other storage applications using the server (1) **616**, the server (N) **618**, etc.

(106) A user interface tier **604** of the distributed computing platform **602** may provide the client device **628** and/or the storage controller **630** with access to user interfaces associated with the storage and access of data and/or other services provided by the distributed computing platform **602**. In an example, a service user interface **610** may be accessible from the distributed computing platform **602** for accessing services subscribed to by clients and/or storage controllers, such as data replication services, application hosting services, data security services, human resource services, warehouse tracking services, accounting services, etc. For example, client user interfaces may be provided to corresponding clients, such as a client (1) user interface **612**, a client (N) user interface **614**, etc. The client (1) can access various services and resources subscribed to by the client (1) through the client (1) user interface **612**, such as access to a web service, a development environment, a human resource application, a warehouse tracking application, and/or other services and resources provided by the application server tier **606**, which may use data stored within the data storage tier **608**.

(107) The client device **628** and/or the storage controller **630** may subscribe to certain types and amounts of services and resources provided by the distributed computing platform **602**. For example, the client device **628** may establish a subscription to have access to three virtual machines, a certain amount of storage, a certain type/amount of data redundancy, a certain type/amount of data security, certain service level agreements (SLAs) and service level objectives (SLOs), latency guarantees, bandwidth guarantees, access to execute or host certain applications, etc. Similarly, the storage controller **630** can establish a subscription to have access to certain services and resources of the distributed computing platform **602**.

(108) As shown, a variety of clients, such as the client device **628** and the storage controller **630**,

incorporating and/or incorporated into a variety of computing devices may communicate with the distributed computing platform **602** through one or more networks, such as the network **626**. For example, a client may incorporate and/or be incorporated into a client application (e.g., software) implemented at least in part by one or more of the computing devices.

(109) Examples of suitable computing devices include personal computers, server computers, desktop computers, nodes, storage servers, storage controllers, laptop computers, notebook computers, tablet computers or personal digital assistants (PDAs), smart phones, cell phones, and consumer electronic devices incorporating one or more computing device components, such as one or more electronic processors, microprocessors, central processing units (CPU), or controllers. Examples of suitable networks include networks utilizing wired and/or wireless communication technologies and networks operating in accordance with any suitable networking and/or communication protocol (e.g., the Internet). In use cases involving the delivery of customer support services, the computing devices noted represent the endpoint of the customer support delivery process, i.e., the consumer's device.

(110) The distributed computing platform **602**, such as a multi-tenant business data processing platform or cloud computing environment, may include multiple processing tiers, including the user interface tier **604**, the application server tier **606**, and a data storage tier **608**. The user interface tier **604** may maintain multiple user interfaces, including graphical user interfaces and/or web-based interfaces. The user interfaces may include the service user interface **610** for a service to provide access to applications and data for a client (e.g., a “tenant”) of the service, as well as one or more user interfaces that have been specialized/customized in accordance with user specific requirements, which may be accessed via one or more APIs.

(111) The service user interface **610** may include components enabling a tenant to administer the tenant's participation in the functions and capabilities provided by the distributed computing platform **602**, such as accessing data, causing execution of specific data processing operations, etc. Each processing tier may be implemented with a set of computers, virtualized computing environments such as a storage virtual machine or storage virtual server, and/or computer components including computer servers and processors, and may perform various functions, methods, processes, or operations as determined by the execution of a software application or set of instructions.

(112) The data storage tier **608** may include one or more data stores, which may include the service data store **620** and one or more client data stores. Each client data store may contain tenant-specific data that is used as part of providing a range of tenant-specific business and storage services or functions, including but not limited to ERP, CRM, eCommerce, Human Resources management, payroll, storage services, etc. Data stores may be implemented with any suitable data storage technology, including structured query language (SQL) based relational database management systems (RDBMS), file systems hosted by operating systems, object storage, etc.

(113) In accordance with one embodiment of the invention, the distributed computing platform **602** may be a multi-tenant and service platform operated by an entity in order to provide multiple tenants with a set of business related applications, data storage, and functionality. These applications and functionality may include ones that a business uses to manage various aspects of its operations. For example, the applications and functionality may include providing web-based access to business information systems, thereby allowing a user with a browser and an Internet or intranet connection to view, enter, process, or modify certain types of business information or any other type of information.

(114) In an embodiment, the described methods and/or their equivalents may be implemented with computer executable instructions. Thus, In an embodiment, a non-transitory computer readable/storage medium is configured with stored computer executable instructions of an algorithm/executable application that when executed by a machine(s) cause the machine(s) (and/or associated components) to perform the method. Example machines include but are not limited to a

processor, a computer, a server operating in a cloud computing system, a server configured in a Software as a Service (SaaS) architecture, a smart phone, and so on). In an embodiment, a computing device is implemented with one or more executable algorithms that are configured to perform any of the disclosed methods.

(115) It will be appreciated that processes, architectures and/or procedures described herein can be implemented in hardware, firmware and/or software. It will also be appreciated that the provisions set forth herein may apply to any type of special-purpose computer (e.g., file host, storage server and/or storage serving appliance) and/or general-purpose computer, including a standalone computer or portion thereof, embodied as or including a storage system. Moreover, the teachings herein can be configured to a variety of storage system architectures including, but not limited to, a network-attached storage environment and/or a storage area network and disk assembly directly attached to a client or host computer. Storage system should therefore be taken broadly to include such arrangements in addition to any subsystems configured to perform a storage function and associated with other equipment or systems.

(116) In some embodiments, methods described and/or illustrated in this disclosure may be realized in whole or in part on computer-readable media. Computer readable media can include processor-executable instructions configured to implement one or more of the methods presented herein, and may include any mechanism for storing this data that can be thereafter read by a computer system. Examples of computer readable media include (hard) drives (e.g., accessible via network attached storage (NAS)), Storage Area Networks (SAN), volatile and non-volatile memory, such as read-only memory (ROM), random-access memory (RAM), electrically erasable programmable read-only memory (EEPROM) and/or flash memory, compact disk read only memory (CD-ROM)s, CD-Rs, compact disk re-writeable (CD-RW)s, DVDs, cassettes, magnetic tape, magnetic disk storage, optical or non-optical data storage devices and/or any other medium which can be used to store data.

(117) Although the subject matter has been described in language specific to structural features or methodological acts, it is to be understood that the subject matter defined in the appended claims is not necessarily limited to the specific features or acts described above. Rather, the specific features and acts described above are disclosed as example forms of implementing at least some of the claims.

(118) Various operations of embodiments are provided herein. The order in which some or all of the operations are described should not be construed to imply that these operations are necessarily order dependent. Alternative ordering will be appreciated given the benefit of this description. Further, it will be understood that not all operations are necessarily present in each embodiment provided herein. Also, it will be understood that not all operations are necessary in some embodiments.

(119) Furthermore, the claimed subject matter is implemented as a method, apparatus, or article of manufacture using standard application or engineering techniques to produce software, firmware, hardware, or any combination thereof to control a computer to implement the disclosed subject matter. The term “article of manufacture” as used herein is intended to encompass a computer application accessible from any computer-readable device, carrier, or media. Of course, many modifications may be made to this configuration without departing from the scope or spirit of the claimed subject matter.

(120) As used in this application, the terms “component”, “module,” “system”, “interface”, and the like are generally intended to refer to a computer-related entity, either hardware, a combination of hardware and software, software, or software in execution. For example, a component includes a process running on a processor, a processor, an object, an executable, a thread of execution, an application, or a computer. By way of illustration, both an application running on a controller and the controller can be a component. One or more components residing within a process or thread of execution and a component may be localized on one computer or distributed between two or more

computers.
(121) Moreover, “exemplary” is used herein to mean serving as an example, instance, illustration, etc., and not necessarily as advantageous. As used in this application, “or” is intended to mean an inclusive “or” rather than an exclusive “or”. In addition, “a” and “an” as used in this application are generally be construed to mean “one or more” unless specified otherwise or clear from context to be directed to a singular form. Also, at least one of A and B and/or the like generally means A or B and/or both A and B. Furthermore, to the extent that “includes”, “having”, “has”, “with”, or variants thereof are used, such terms are intended to be inclusive in a manner similar to the term “comprising”.
(122) Many modifications may be made to the instant disclosure without departing from the scope or spirit of the claimed subject matter. Unless specified otherwise, “first,” “second,” or the like are not intended to imply a temporal aspect, a spatial aspect, an ordering, etc. Rather, such terms are merely used as identifiers, names, etc. for features, elements, items, etc. For example, a first set of information and a second set of information generally correspond to set of information A and set of information B or two different or two identical sets of information or the same set of information.
(123) Also, although the disclosure has been shown and described with respect to one or more implementations, equivalent alterations and modifications will occur to others skilled in the art based upon a reading and understanding of this specification and the annexed drawings. The disclosure includes all such modifications and alterations and is limited only by the scope of the following claims. In particular regard to the various functions performed by the above described components (e.g., elements, resources, etc.), the terms used to describe such components are intended to correspond, unless otherwise indicated, to any component which performs the specified function of the described component (e.g., that is functionally equivalent), even though not structurally equivalent to the disclosed structure. In addition, while a particular feature of the disclosure may have been disclosed with respect to only one of several implementations, such feature may be combined with one or more other features of the other implementations as may be desired and advantageous for any given or particular application.

Claims

1. A method comprising: populating snapshot data into a plurality of slots of objects that are stored into an object store, wherein the snapshot data is represented accordingly to an object file system that structures the snapshot data according to a tree structure comprising a plurality of nodes; receiving a request to access a portion of the snapshot data; traversing the plurality of nodes of the tree structure to extract a cloud block number comprising a sequence number assigned to an object and a slot number of a slot of the object storing the portion of the snapshot data, wherein the tree structure includes a node storing the cloud block number comprising the sequence number and the slot number; and utilizing the sequence number and slot number extracted from the node of the tree structure to access to the portion of the snapshot data stored within the slot of the object.
2. The method of claim 1, comprising: storing inofile blocks, of an inofile comprising per inode metadata, into slots of the object.
3. The method of claim 1, comprising: storing at least one of user data, directory blocks, metadata, or inofile blocks of an inofile comprising per inode metadata into slots of the object.
4. The method of claim 3, comprising: representing the at least one of the user data, the directory blocks, the metadata, or the inofile blocks as the plurality of nodes within the tree structure.
5. The method of claim 1, comprising: storing user data within leaf nodes of the tree structure and pointers within non-leaf nodes of the tree structure, wherein an inode object, for a file, comprises pointers that point to non-leaf nodes within a top level of the tree structure.
6. The method of claim 5, comprising: traversing the inode object and the tree structure to identify the sequence number and the slot number of the slot storing the portion of the snapshot data.

7. The method of claim 1, comprising: storing a plurality of snapshots, maintained by a computing device, within objects stored in the object store, wherein each snapshot is identifiable through a snapinfo object, wherein each snapinfo object has a unique generation number.
8. The method of claim 1, comprising: creating an object header for the object, wherein the object header comprises a slot context for the plurality of slots within the object.
9. The method of claim 8, wherein the slot context comprises a compression type, a start offset of slots, a logical data length, normalization data, decoding information, and a compressed data length.
10. The method of claim 9, wherein the slot context is used to access compressed data, compressed by a computing device, stored within the object.
11. A computing device comprising: a memory comprising machine executable code for performing a method; and a processor coupled to the memory, the processor configured to execute the machine executable code to cause the processor to perform operations comprising: populating snapshot data into a plurality of slots of objects that are stored into an object store, wherein the snapshot data is represented accordingly to an object file system that structures the snapshot data according to a tree structure comprising a plurality of nodes; receiving a request to access a portion of the snapshot data; traversing the plurality of nodes of the tree structure to extract a cloud block number comprising a sequence number assigned to an object and a slot number of a slot of the object storing the portion of the snapshot data, wherein the tree structure includes a node storing the cloud block number comprising the sequence number and the slot number; and utilizing the sequence number and slot number extracted from the node of the tree structure to access to the portion of the snapshot data stored within the slot of the object.
12. The computing device of claim 11, wherein the operations comprise: storing inofile blocks, of an inofile comprising per inode metadata, into slots of the object.
13. The computing device of claim 11, wherein the operations comprise: storing at least one of user data, directory blocks, metadata, or inofile blocks of an inofile comprising per inode metadata into slots of the object.
14. The computing device of claim 13, wherein the operations comprise: representing the at least one of the user data, the directory blocks, the metadata, or the inofile blocks as the plurality of nodes within the tree structure.
15. The computing device of claim 11, wherein the operations comprise: storing user data within leaf nodes of the tree structure and pointers within non-leaf nodes of the tree structure, wherein an inode object, for a file, comprises pointers that point to non-leaf nodes within a top level of the tree structure.
16. The computing device of claim 15, wherein the operations comprise: traversing the inode object and the tree structure to identify the sequence number and the slot number of the slot storing the portion of the snapshot data.
17. A non-transitory machine readable medium comprising instructions for performing a method, which when executed by a machine, causes the machine to perform operations comprising: populating snapshot data into a plurality of slots of objects that are stored into an object store, wherein the snapshot data is represented accordingly to an object file system that structures the snapshot data according to a tree structure comprising a plurality of nodes; receiving a request to access a portion of the snapshot data; traversing the plurality of nodes of the tree structure to extract a cloud block number comprising a sequence number assigned to an object and a slot number of a slot of the object storing the portion of the snapshot data, wherein the tree structure includes a node storing the cloud block number comprising the sequence number and the slot number; and utilizing the sequence number and slot number extracted from the node of the tree structure to access to the portion of the snapshot data stored within the slot of the object.
18. The non-transitory machine readable medium of claim 17, wherein the operations comprise: storing inofile blocks, of an inofile comprising per inode metadata, into slots of the object.

19. The non-transitory machine readable medium of claim 17, wherein the operations comprise:
storing at least one of user data, directory blocks, metadata, or inofile blocks of an inofile comprising per inode metadata into slots of the object.
20. The non-transitory machine readable medium of claim 19, wherein the operations comprise:
representing the at least one of the user data, the directory blocks, the metadata, or the inofile blocks as the plurality of nodes within the tree structure.